WB Brands Finance OS

Feature Status & Engineering Health Report

A product-engineering snapshot of every feature shipped to date, the stack underneath it, and a candid log of what is partially built, deferred, or known to be brittle. Intended for stakeholders and incoming engineers who need a single source of truth on where the platform actually stands.

Branch	nextjs-migration
Prepared	2026-05-14
Audience	Leadership, Engineering, Onboarding
Author	Product Engineering

This document is auto-generated from a codebase audit. Status flags reflect what is verifiable in the repo today (routes, server actions, queries, migrations) - not aspirational scope.

TL;DR

WB Brands Finance OS is a Next.js 15 / React 19 / Supabase rewrite of the legacy vanilla-JS finance dashboard. The migration is functionally complete: 23 of 24 user-facing pages are shipped, all 21 server actions are wired, and the auth model (4 roles, page + action gates) is enforced end to end. The one feature actively in flight is the P&L refactor (new account-subtype structure under migration 0007).

The biggest open risk is that Row-Level Security policies exist in the database but are not enforced - authorisation lives entirely in the app layer today. That is the headline Phase-2 item. Secondary gaps: no automated test suite, in-memory rate limiting only, and the Admin-API data sync is scaffolded but not wired to any UI yet.

At a glance

- Pages shipped: 23 / 24 (P&L refactor in progress)
- Server Actions: 21, all role-gated, all audit-logged
- Query modules: 13, covering every domain surface
- Database migrations: 7 (0007 untracked, pending commit)
- Auth: cookie SSR via @supabase/ssr, 4 roles, full matrix
- AI advisor: live (Anthropic Claude Haiku), 20 req/min/user
- RLS: written but NOT enforced - app-layer auth only
- Automated tests: none (Playwright + Vitest planned)

1. Stack & Architecture

- Next.js 15 App Router, React 19, TypeScript strict mode
- Tailwind CSS v4 (CSS-first @theme in app/globals.css)
- Supabase Postgres + Auth, accessed via @supabase/ssr cookie sessions
- Zod for runtime validation of env + payloads
- Anthropic Claude Haiku for the AI advisor (server-side route handler)
- Hosted on Vercel; Edge middleware refreshes Supabase JWT on every request
- All pages are Server Components with export const dynamic = 'force-dynamic'

2. Shipped Features by Page

Each entry below is a real route under app/(app)/ with its current production status, the role gate that protects it, and what it does for the user. Status flags: SHIPPED = working in main flows; IN PROGRESS = code is on disk but actively being reworked; DEFERRED = scaffolded, not wired to UI.

Overview & Dashboard

Dashboard /dashboard **SHIPPED**

KPI cards (revenue, gross & net profit, cash position, overdue invoices), P&L summary, transaction feed grouped by cash account section. COO + Admin.

Cash Forecast /forecast **SHIPPED**

13-week rolling forecast. Starting cash pulled from cash_balances, weekly inflow/outflow extrapolated from the trailing-30-day run-rate. COO + Admin.

Ratios & KPIs /ratios **SHIPPED**

Current ratio, debt-to-equity, gross & net margin, EBITDA, YTD balance-sheet approximations. COO + CPA + Admin.

Accounting Core

Bank Transactions (Inbox) /inbox **SHIPPED**

Unclassified bank txns with auto-tagging via classification_rules. Detects entity, transaction kind (transfer / payment / deposit), supports classify, split, edit, delete. Bookkeeper + Admin.

Credit Card Inbox /cc-inbox **SHIPPED**

Parallel inbox for CC rows; shares the InboxClient component. Bookkeeper + Admin.

Ledger /ledger **SHIPPED**

Journal view of every posted txn (bank + JE-tagged) for the active period. Sortable by account / date / amount. COO + Bookkeeper + Admin.

Journal Entries /journals **SHIPPED**

Manual JE CRUD, month close, month reopen. Merges explicit JE rows with JE-tagged transactions. Monthly selector. COO + Bookkeeper + CPA + Admin.

Reconciliation /reconcile **SHIPPED**

Two-column bank-vs-book match. Auto-match by amount (+/- $0.01) and date (+/- 3 days). Manual override and unmatch. COO + Bookkeeper + Admin.

Chart of Accounts /coa **SHIPPED**

Account master with live period balances. Create / update / deactivate accounts. Bookkeeper + CPA + Admin.

Payables & Procurement

Vendors	/vendors	SHIPPED

Vendor master CRUD; consumed by invoice and AP screens. COO + Bookkeeper + Admin.

Invoices	/invoices	SHIPPED

Vendor invoices with payment recording and status tracking (open / partial / overdue / paid); links to AP items. COO + Bookkeeper + Admin.

AP / Payables	/ap	SHIPPED

Open AP items with vendor + invoice detail. KPI cards (total due, overdue, due-this-week, avg days outstanding). Pay / dispute actions. COO + CPA + Admin.

Financial Reports

Profit & Loss	/pnl	IN PROGRESS

Sectioned (Gross Revenue, COGS, Marketing, OpEx) with entity-column drilldown, collapsible by account_subtype. Account structure is mid-refactor under migration 0007. COO + CPA + Admin.

Balance Sheet	/balance	SHIPPED

Assets / Liabilities / Equity by account; YTD retained earnings; balance check. COO + CPA + Admin.

Cash Flow	/cashflow	SHIPPED

Indirect-method statement with operating / investing / financing buckets, mapped heuristically by account_subtype. COO + CPA + Admin.

Cash Balances	/cash-balances	SHIPPED

Manual cash position snapshot by entity and column key (tfb, hunt, vend_pay, cc, ...); tracks updated_at. COO + Bookkeeper + CPA + Admin.

Cashbook	/cashbook	SHIPPED

Payment-method and sales-summary snapshots for a date range; generates journals from snapshots. COO + CPA + Admin.

CFO Notes	/cfnotes	SHIPPED

Period-scoped narrative notes (create / edit / delete). COO + CPA + Admin.

Sales & Product

Sales Metrics	/sales	SHIPPED

Daily revenue bucketing; line chart by day; count + total stats. COO + Admin.

Product Mix	/productmix	SHIPPED

Revenue by account (product / category view); bars + table. COO + Admin.

Setup & Admin

Bank Connections	/banks	SHIPPED

Bank account master (institution, account name, last 4). CRUD. COO + Admin.

Import Data	/import	SHIPPED

CSV / XLSX upload, entity detection from bank-account name, preview-then-commit flow, mirrors legacy parser. All roles.

Users (Admin)	/admin/users	SHIPPED

List, invite and role-assign users via Supabase admin API. Degrades gracefully if the service-role key is absent. Admin only.

Classification Rules (Admin)	/admin/rules	SHIPPED

Regex / substring rules that auto-tag bank + CC txns. Create / edit / delete, with bulk-apply to unclassified rows. Admin only.

Authentication

Login	/login	SHIPPED

Email + password sign-in, redirects to each role's landing page on success.

No-Role Bootstrap	/no-role	SHIPPED

First user in a fresh tenant can claim admin if zero admins exist; otherwise prompts to contact an admin.

3. Server Actions (Domain Mutations)

All mutations go through Server Actions in /actions. Every action calls requireRole() and writes to audit_log via writeAuditLog(). Audit logging no-ops gracefully if the table is absent.

transactions.ts

- Functions: classifyTransaction, bulkClassifyTransactions, splitTransaction, markAsInternalTransfer, markAsCcPayment, deleteRawTransaction, editTransaction
- Roles: bookkeeper, admin

classify.ts

- Functions: upsertClassificationRule, bulkAutoTag, deleteClassificationRule
- Roles: coo, bookkeeper, admin

reconcile.ts

- Functions: markMatched, autoMatchPeriod, unmatch
- Roles: coo, bookkeeper, admin

journals.ts

- Functions: createJournal, deleteJournal, closeMonth, reopenMonth
- Roles: coo, bookkeeper, cpa, admin

invoices.ts

- Functions: createInvoice, recordPayment, deleteInvoice
- Roles: coo, bookkeeper, admin

vendors.ts

- Functions: createVendor, updateVendor, deleteVendor
- Roles: coo, bookkeeper, admin

ap.ts

- Functions: payApItem, disputeApItem
- Roles: coo, cpa, admin

cash.ts

- Functions: saveCashBalance
- Roles: coo, cpa, admin

accounts.ts

- Functions: createAccount, updateAccount, deactivateAccount
- Roles: coo, bookkeeper, admin

banks.ts

- Functions: createBankConnection, updateBankConnection, deleteBankConnection
- Roles: coo, admin

cashbook.ts

- Functions: refreshCashbookSnapshot, generateCashbookJournals
- Roles: coo, cpa, admin

import.ts

- Functions: previewImport, commitImport
- Roles: all roles

cfnotes.ts

- Functions: createCfoNote, updateCfoNote, deleteCfoNote
- Roles: coo, cpa, admin

period-close.ts

- Functions: closeMonthWithAdjustments
- Roles: coo, bookkeeper, admin

reports.ts

- Functions: drillDownAccount
- Roles: coo, cpa, admin

4. Role-Based Access Control

Four roles, defined once in lib/auth/permissions.ts and enforced in three places: Edge middleware, <PageShell> (page-level), and <RoleGate> (per-action UI).

Roles & landing pages

- COO -> /dashboard
- Bookkeeper -> /inbox
- CPA -> /pnl
- Admin -> /dashboard

Sidebar groups (visibility is role-gated per page)

- Overview: Dashboard
- Accounting: Inbox, CC-Inbox, Ledger, Journals, Reconcile, COA, Admin-Rules
- Payables: Vendors, Invoices, AP
- Reports: P&L, Balance, Cashflow, Forecast, Cashbook, Cash-Balances, Ratios, CFO Notes
- Sales: Sales Metrics, Product Mix
- Setup: Banks, Import
- Admin: Users

Discrete actions (not pages)

- sync-sheets -> COO
- add-transaction -> Bookkeeper
- clear-all-data -> COO, Bookkeeper, Admin
- ai-advisor -> COO
- dashboard-runway-card -> COO

5. Entities

All financial data is scoped by entity. Codes: WB (WB Brands LLC), WBP (WB Promo), LP (Lanyard Promo), KP (Koolers Promo), BP (Band Promo), SWAG (Swagprint), RUSH, ONEOPS (One Operations Mgmt), SP1.

Import flow auto-detects entity from the source bank-account name via detectEntityFromBankAccount() in lib/entities.ts. P&L reports use the PNL_ENTITY_COLUMNS layout to render each entity as a column.

6. AI Advisor

- Route: POST /api/ai, COO-only via canDoAction('ai-advisor')
- Model: Anthropic Claude Haiku
- Context: live financial state (period + entity) is injected per request
- Rate limit: 20 req/min/user (in-memory, lost on cold start)
- UI: AdvisorPanel.tsx in the topbar

7. Data Layer

Three Supabase clients live in lib/supabase: server.ts (cookie-bound, used for auth flows), middleware.ts (Edge session refresh), and data.ts (anon-key client for app-layer queries). Types are generated into lib/supabase/types.ts via supabase gen types.

Query helpers live in lib/queries/* and return domain-shaped rows with the joins already resolved. All reads filter by entity scope and the period parsed from searchParams (lib/period.ts).

8. Database Migrations

0001_baseline_schema.sql

- Status: Applied
- Core tables: transactions, journal_entries, ledger_entries, accounts, entities, vendors, invoices, ap_items, raw_transactions, bank_connections, cash_balances, classification_rules, cashbook_snapshots, cfo_notes, reconciliation_matches

0002_enable_rls.sql

- Status: Applied (NOT ENFORCED)
- ALTER TABLE ... ENABLE ROW LEVEL SECURITY with default-deny policies. Real per-role policies are written but the app still bypasses them via the anon-key client - see Section 11.

0003_user_profiles.sql

- Status: Applied
- profiles table (user_id PK, role enum, email, display_name) plus seed users.

0004_audit_log.sql

- Status: Applied
- audit_log table (id, actor_user_id, table_name, row_id, op, before jsonb, after jsonb, at). Append-only.

0005_cashbook_snapshots.sql

- Status: Applied
- Expanded cashbook_snapshots for payment-method & sales-summary storage.

0006_admin_api_accounts.sql

- Status: Applied
- Admin-API account mapping tables. Scaffolding only - not wired to any UI yet.

0007_pnl_account_structure.sql

- Status: UNTRACKED
- Refactor of account_subtype to support new P&L sections (gross_revenue, sales_return, platform_fee, cogs, sales_tax, marketing, labour, opex, distribution). On disk, not yet committed.

9. Documentation & Tests

- CLAUDE.md: comprehensive project guide with phase plan - up to date
- docs/nextjs-migration-plan.md: 5-phase plan, currently post-Phase-1
- docs/superpowers/: per-sprint design notes and audits
- Playwright E2E suite: NOT YET WRITTEN
- Vitest unit suite: NOT YET WRITTEN

10. What's Broken, Incomplete, or Brittle

Honest list. Some items are tracked work that is mid-flight; others are scaffolding that has never been wired up; a few are real risks that warrant attention before we scale users.

10.1 In-flight (modified but not yet committed)

These files show as M in git status on the nextjs-migration branch:

- actions/classify.ts - classification-rule logic enhancements
- actions/transactions.ts - new split / edit / mark-internal-transfer capabilities
- app/(app)/pnl/page.tsx + PnlClient.tsx - major P&L refactor with new account structure
- app/(app)/inbox/InboxClient.tsx + page.tsx - improved classification UX
- app/(app)/cc-inbox/page.tsx - parallel CC inbox tweaks
- lib/auth/permissions.ts - role matrix tweaks
- lib/classify-rules.ts - classification engine enhancements
- lib/entities.ts - entity defs update
- lib/format.ts - formatting helpers
- lib/period.ts - period calc fixes
- lib/queries/reports.ts - report query enhancements
- supabase/migrations/0007_pnl_account_structure.sql - new migration, untracked

Risk: the working tree has a non-trivial amount of uncommitted work. Any deploy from this branch should happen on a clean commit, not directly from the working copy.

10.2 Database / Security

RLS exists but is not enforced (HIGH)

- Policies live in migration 0002 but the app reads via the anon-key client.
- Net effect: any authenticated session could query any row by hitting PostgREST directly.
- All authorisation currently lives in middleware + requireRole() + <RoleGate>.
- This is the headline Phase-2 work in docs/nextjs-migration-plan.md.

Audit log path is not yet load-tested

- writeAuditLog() runs synchronously inside each server action and no-ops on table-missing.
- We have no monitoring on audit_log volume or query performance yet.

10.3 Features that are scaffolded but not wired

Admin API integration

- lib/admin-api/* has client, token refresh, schemas, journal + entity mappers, and a reports module.
- No UI consumes it; 0006 migration prepared the schema. Currently dead code from a user POV.

Persistent AI advisor history

- Each /api/ai request is independent; no thread is persisted in Postgres.
- Rate limit is in-memory only, so a Vercel cold start resets a user's quota.

Realtime updates

- Not implemented. Pages use force-dynamic SSR; updates require a refresh.
- Acceptable for current scale but worth flagging for the inbox + reconcile screens.

10.4 Reporting accuracy caveats

Cash Forecast is naive

- Uses a trailing-30-day average for both inflow and outflow projection.
- There is no assumptions table, no manual override, and no seasonality handling.

Cash Flow buckets are heuristic

- Operating / investing / financing classification is inferred from account_subtype.
- Edge accounts can land in the wrong bucket; a manual override mechanism is not yet built.

Single-currency

- Everything is treated as USD. No FX, no multi-currency reporting.

No budgets

- There is no budgets table and no budget-vs-actual variance reporting.

10.5 Operational gaps

- No automated test coverage at all (no Playwright, no Vitest, no API contract tests).
- Rate limiting is in-process only; needs Upstash/Redis to be reliable behind a load balancer.
- No structured logging or error monitoring is wired in (Sentry, Logtail, etc).
- Cashbook snapshots assume manual insertion - no fetcher from payment processors.
- First-user-claims-admin bootstrap (/no-role) works but has no audit trail of the claim event.

11. Recommended Next Phase

Ordered by user-visible impact relative to engineering cost. Items 1-3 are pre-requisites for opening the app to a broader internal audience.

- 1. Commit the in-flight P&L refactor (0007 migration + modified files) once entity-team confirms account_subtype mapping is complete.
- 2. Enforce the existing RLS policies and switch the data client off the anon key for authenticated reads. Tighten the policy matrix to match lib/auth/permissions.ts.
- 3. Stand up Playwright E2E covering login -> inbox -> classify -> reconcile -> close-month. This is the single highest-leverage regression catcher.
- 4. Move the AI advisor rate-limit and any future per-user counters to Upstash/Redis.
- 5. Wire the Admin-API client into a sync screen so the scaffolding under lib/admin-api/* starts earning its keep.
- 6. Replace the trailing-30-day forecast with an assumptions table and a manual-override UI.
- 7. Add Sentry (or equivalent) for unhandled server-action errors and middleware crashes.

Closing note

From a feature standpoint the platform is in a strong place: the legacy tool's full surface area has been re-implemented with proper role gating, audit logging, and a real auth model. The work that remains is mostly hardening (RLS, tests, observability) and finishing a couple of scaffolded integrations - not net-new product surface.